# Genetic Sequence Motif Finder

GURALA SIVA TEJA REDDY
*M.Tech, Computer Science and Engineering*
*Koneru Lakshmaiah Education Foundation*
2601050128@kluniversity.in

Raju Anitha
*Professor*
*Department of Computer Science and Engineering*
*Koneru Lakshmaiah Education Foundation*
anitharaju@kluniversity.in

**Abstract—**
**Recurring sequence patterns are useful starting points for exploring conserved regions in DNA and protein data. This paper presents Genetic Sequence Motif Finder, an educational sequence-analysis framework that combines exact k-mer indexing, divide-and-conquer scanning, multiprocessing, frequency-based ranking, Longest Common Subsequence (LCS) similarity, and Smith–Waterman local alignment. The input pipeline parses FASTA records, normalizes and validates sequence symbols, and creates tasks over valid k-mer start positions. Workers build partial motif maps that are merged into a position-aware index, allowing the system to report occurrence counts, sequence support, and 1-based locations. A command-line interface and a local browser dashboard expose search and comparison workflows, while CSV and JSON exports support reuse. The project documentation reports 20 passing unit tests and includes a synthetic experiment with 30 DNA sequences of 1,000 residues each. In that example, ATGCGTAC was reported 51 times across 25 sequences and CGCGAATT 40 times across 21 sequences. These results demonstrate recovery of planted substrings in a controlled dataset, not biological validation or general performance guarantees. The paper describes the architecture, algorithmic choices, evaluation evidence, limitations, and directions for future work.**

**Index Terms—**Bioinformatics, DNA sequence, k-mer indexing, motif discovery, multiprocessing, LCS, Smith–Waterman alignment.

## I. INTRODUCTION

Biological sequence analysis represents DNA and proteins as strings over finite alphabets. A short recurring substring can be a useful candidate for further investigation because conserved sequence regions may be associated with binding, regulation, or other biological processes. However, recurrence alone does not establish biological function. Motif discovery methods differ in whether they search exact strings, allow substitutions, or estimate statistical significance. Surveys of motif-finding algorithms describe consensus, probabilistic, combinatorial, and nature-inspired approaches and emphasize trade-offs among accuracy, search cost, and assumptions [1].

Genetic Sequence Motif Finder focuses on an interpretable computational baseline: exact k-mer counting with position tracking, combined with sequence-comparison algorithms. For a sequence of length n, a fixed k produces $\max(0, n-k+1)$ overlapping windows. Indexing these windows enables frequency and sequence-support calculations, while the recorded positions make results auditable. K-mer indexing is widely studied for querying large sequencing collections, and published work discusses both the advantages and limitations of alternative data structures [2].

The contribution of this project is an integrated educational workflow that connects preprocessing, exact motif indexing, safe chunking, parallel execution, result ranking, LCS similarity, local alignment, and user-facing reporting. It does not claim a new motif-discovery theory or replace statistical tools such as STREME [3] or enrichment systems such as SEA [4]. Instead, it provides a transparent platform for inspecting the behavior and interaction of standard algorithms.

## II. RELATED WORK

### A. Motif Discovery and Enrichment

Hashim et al. classify motif discovery approaches and discuss the limitations of methods that may be computationally expensive or sensitive to local optima [1]. STREME was presented as an accurate and versatile sequence motif discovery method with statistical significance estimates and support for DNA, RNA, protein, and user-defined alphabets [3]. SEA addresses motif enrichment, a different question from counting exact recurring strings: enrichment analysis evaluates whether a motif is over-represented under a background model [4]. These systems motivate an important distinction in the present work: the engine returns exact recurrence candidates, but it does not assign biological significance or statistical enrichment scores.

### B. K-mer Indexing

Marchet et al. survey k-mer-based data structures for querying large sequencing collections and describe how index representations influence storage, query performance, and scale [2]. BLight proposes an efficient exact associative structure for k-mers [5], while Themisto uses a scalable colored k-mer index for sensitive pseudoalignment across very large collections of bacterial genomes [6]. The masked Burrows–Wheeler-transform-based FMSI approach targets space-efficient indexing for unconstrained k-mer sets [7]. Roy and Mukhopadhyay investigate randomized optimal k-mer indexing for parallel genome sequence compression [8].

These systems target scale and specialized query requirements. The present implementation uses ordinary hash maps because they are readily inspectable and suitable for a standard-library Python project. It also stores each occurrence position, which increases memory consumption compared with indexes designed only for membership or compact

lookup. The reviewed work suggests future directions such as packed encodings, compressed indexes, and disk-backed storage.

### C. Sequence Alignment and Parallel Computation

LCS is a dynamic-programming method for measuring the longest order-preserving subsequence shared by two strings. Kumar and Manna describe a graphical interface for visualizing LCS behavior in DNA sequence comparison [9]. Singh et al. discuss a dynamic MPI-based memory-efficient framework for LCS on massive DNA sequences [10], while Kong et al. examine adaptive parallel computation for large-scale multiple LCS problems [11]. These works show that the LCS recurrence is conceptually straightforward but can become expensive as input sizes increase.

Smith–Waterman computes an optimal local alignment under a selected scoring scheme. Pałkowski and Bielecki investigate parallel tiled implementations for two- and three-sequence alignment [12]. Muhamad et al. compare Needleman–Wunsch global alignment and Smith–Waterman local alignment and discuss parallel execution [13]. Seliem et al. combine divide and conquer, MapReduce, and FPGA acceleration for chromosome sequence mapping [14]. Rafati Bonab et al. explore actor-based acceleration of Smith–Waterman [15]. Parallel sequence search and multiple alignment have also been studied through heterogeneous computation and alternative alignment strategies [16], [17], [18]. This project uses baseline dynamic-programming implementations and process-level parallelism for independent motif-search chunks, not specialized hardware acceleration.

## III. PROBLEM DEFINITION AND OBJECTIVES

Given a collection of DNA or protein sequences, the system must find exact substrings within a configurable k range, count all occurrences, report the sequence identifiers and positions, and rank candidates according to occurrence frequency and sequence support. It must also support pairwise sequence comparison through LCS similarity and Smith–Waterman local alignment. The same core search semantics should apply to sequential and multiprocessing modes.

The main correctness challenge in chunked scanning is avoiding lost or duplicated motifs at partition boundaries. If arbitrary text slices are scanned independently, a k-mer crossing a split may be omitted. This implementation partitions valid start positions and makes the required trailing residues available to each chunk. Each start position is owned by one task, so merging partial maps should preserve exact counts.

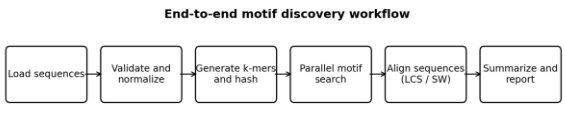



Fig. 2. End-to-end sequence motif discovery workflow.

The project objectives are: (1) parse and validate FASTA records; (2) index k-mers and preserve 1-based positions; (3) support divide-and-conquer and multiprocessing scans; (4) filter and rank motifs; (5) compute LCS similarity and local alignments; (6) provide CLI and browser-based workflows; and (7) verify behavior with unit tests and sequential/parallel equivalence checks.

## IV. SYSTEM ARCHITECTURE

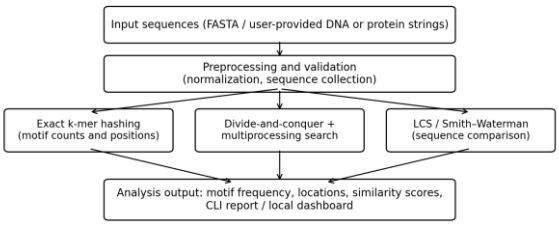



Fig. 1. System architecture of the Genetic Sequence Motif Finder.

The processing pipeline comprises five logical stages. First, the input layer accepts FASTA records or pasted sequence text. Second, preprocessing normalizes case and whitespace, applies the documented rules for gaps and digits, detects the sequence alphabet, and validates records. Third, the search planner creates disjoint ranges of valid k-mer start positions. Fourth, worker tasks build partial maps keyed by motif and sequence identifier; the parent process merges these maps and sorts their position lists. Fifth, the analysis layer applies thresholds, ranks results, optionally runs alignment, and exports the output.

| Module | Primary responsibility |
|---|---|
| preprocessing.py | FASTA parsing, normalization, alphabet detection and validation |
| motif_search.py | Exact k-mer extraction and hash-based indexing |
| divide_conquer.py | Recursive start-position partitioning and map merging |
| multiprocessing_search.py | Chunk scheduling, process pool and sequential comparison |
| sequence_alignment.py | LCS, similarity matrix and Smith–Waterman traceback |
| analysis.py | Filtering, ranking, formatting and CSV/JSON export |
| main.py / server.py | CLI and local web API/dashboard |

The modular design separates data validation from search logic and presentation. The command-line interface and local dashboard call the same core engine, which reduces the risk of inconsistent results across interfaces. The dashboard presents summary metrics, ranked motifs, position information, similarity results, and export options. The web service is intended for local educational use; public deployment would require additional input limits, security controls, logging, and operational hardening.

## V. ALGORITHMS AND IMPLEMENTATION

### A. Exact k-mer Hashing

For each sequence S and each requested k, the scanner visits every valid start i and extracts S[i:i+k]. The motif is used as a hash-map key, and the corresponding sequence identifier and position are appended to its occurrence list. If internal offsets are zero-based, the report converts position i to i+1. Overlapping occurrences are retained. For a single k, the scan performs O(n) window operations; for K requested lengths, work is proportional to the total number of valid windows over those lengths. Memory is O(U+P), where U is the

number of distinct motifs and P is the number of stored occurrences.

### B. Divide and Conquer

The scanner receives a half-open range of valid start positions. If the range is small enough, it scans directly; otherwise, it splits the range into two subranges, recursively scans each, and merges their partial maps. Since subranges are disjoint, every valid start position is processed once. The implementation must retain enough trailing sequence characters to construct windows near a chunk end, but it must not count starts outside the task's assigned interval.

### C. Multiprocessing

Independent chunk tasks can be dispatched to a process pool. Each worker returns a partial motif map; the parent merges maps using the same rules as the sequential path. Process-level parallelism can use multiple CPU cores for CPU-bound tasks, but task serialization, process startup, scheduling, and result merging add overhead. Therefore, parallel execution is not guaranteed to improve performance for small datasets. The benchmark compares identical workloads and verifies output equivalence before interpreting elapsed time.

### D. LCS Similarity and Smith–Waterman

For strings A and B, the LCS dynamic-programming table L uses $L[i,j]=L[i-1,j-1]+1$ when the current symbols match and $\max(L[i-1,j],L[i,j-1])$ otherwise. The reported normalized similarity is $2L/(|A|+|B|)$, which ranges from zero to one for non-empty inputs. The full table requires O(nm) time and memory. Smith–Waterman uses a local score recurrence with zero as a candidate, allowing negative-scoring prefixes to be discarded. The supplied implementation uses a simple match/mismatch/gap scoring scheme and traceback from the highest-scoring cell. Its scores depend on those parameters and should not be treated as universal biological measures.

### E. Result Filtering and Export

The analysis layer calculates total occurrences and sequence support, applies minimum-frequency and minimum-support thresholds, and sorts candidates. An optional maximal-motif filter reduces some redundant substring reports according to the implementation's rule. CSV export supports spreadsheet analysis, while JSON can preserve nested identifiers and position lists. The tool reports exact string recurrence rather than statistical significance.

## VI. EXPERIMENTAL RESULTS

### A. Test Coverage

The supplied project documentation reports 20 passing unit tests. The documented coverage includes FASTA parsing and validation, plain-scan versus divide-and-conquer equivalence at multiple leaf sizes, chunk-boundary handling with small chunks, sequential/parallel result equivalence, the bundled motif example, LCS and Smith–Waterman behavior, and web API checks. These are reported project results; the experiments were not independently rerun while preparing this paper.

### B. Bundled Sequence Example

The project documentation reports that the motif ATGCGT occurs five times across three short sequences. GCGT is also reported five times across three sequences, while GATG occurs three times across three sequences. The position-aware index makes it possible to distinguish repeated hits within a record from a motif distributed across multiple records. These values are examples from the supplied project documentation and are not biological findings.

| Motif | Occurrences | Sequence support |
|---|---|---|
| ATGCGT | 5 | 3 sequences |
| GCGT | 5 | 3 sequences |
| GATG | 3 | 3 sequences |

### C. Synthetic Planted-Motif Experiment

The supplied documentation describes 30 synthetic DNA sequences of 1,000 residues each, for 30,000 residues in total. The search used k values from 4 to 8, maximal-motif filtering, and a minimum support threshold of 10 sequences. ATGCGTAC was reported 51 times across 25 sequences, while CGCGAATT was reported 40 times across 21 sequences. Shifted overlapping strings such as TATGCGTA, TGCGTACT, and TGCGTACC also appeared among the leading results.

| Motif | Occurrences | Sequences |
|---|---|---|
| ATGCGTAC | 51 | 25 |
| CGCGAATT | 40 | 21 |
| TATGCGTA | 22 | reported top result |
| TGCGTACT | 18 | reported top result |
| TGCGTACC | 15 | reported top result |

Recovery of planted substrings is evidence that the exact-counting workflow behaves as intended on this synthetic input. Related shifted substrings can also be frequent because overlapping windows share residues with the planted pattern. This does not establish motif enrichment, biological function, or generalization to real sequencing data.

### D. Similarity and Local Alignment

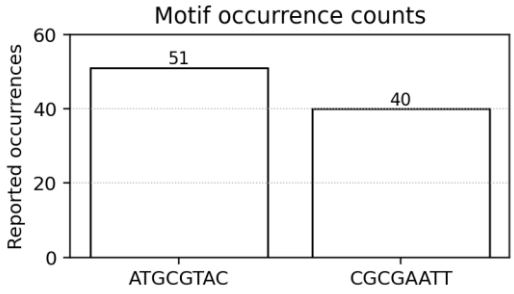



Fig. 3. Reported occurrences of two planted motifs in the synthetic experiment.

The project documentation reports off-diagonal LCS similarity values of 0.73, 0.84, and 0.65 for the three-sequence example. It also reports a Smith–Waterman score of 23 and 87% identity for one local alignment, and a shared segment GATGCGT with 100% identity for another pair. These values depend on the selected sequences and alignment scoring scheme. LCS similarity and local identity measure different properties and should not be interpreted interchangeably.

### E. Performance Interpretation

The benchmark is designed to compare sequential and process-pool elapsed time, calculate speedup as Tseq/Tpar, calculate efficiency as speedup divided by worker count, and check output equality. The project documentation notes that parallel execution can be slower on a single-core machine or for small inputs because overhead dominates. No universal speedup is claimed because complete hardware details and repeated timing distributions are not provided. A stronger evaluation should record processor, memory, Python version, chunk size, worker count, input size, and repeated-trial statistics.

## VII. DISCUSSION

The system integrates exact recurrence, position tracking, sequence comparison, and user-facing reporting into a single workflow. Its strengths are transparency, a modular implementation, reproducible synthetic examples, and the ability to compare sequential and parallel execution. The literature on large-scale k-mer indexes shows that compact representations become important as datasets grow [2], [5]–[8]. Likewise, research on LCS and Smith–Waterman acceleration demonstrates that dynamic programming can become a bottleneck for large inputs [10]–[15].

The principal limitations follow from the selected baseline algorithms. Storing all positions can consume substantial memory; the number of distinct k-mers grows with the input and k range. Full dynamic-programming tables require quadratic space in the lengths of the two sequences. Exact k-mer counts do not tolerate substitutions and do not estimate motif significance. The simple local-alignment scoring scheme is not a substitute for domain-specific substitution matrices or affine gap penalties. Finally, synthetic data offers controlled validation but cannot reproduce the complexity of biological datasets.

Future work should investigate packed or rolling k-mer encodings, compressed indexes, streaming/external-memory processing, statistical motif discovery and enrichment, ambiguity-aware sequence policies, affine-gap alignment, and optimized parallel alignment. Evaluation should compare the tool with established methods on curated datasets and report sensitivity, precision, memory use, runtime, and parameter settings. These extensions are consistent with the directions represented in the supplied literature, but are not claimed as completed features.

## VIII. CONCLUSION

This paper presented Genetic Sequence Motif Finder, a Python-based framework for exact k-mer motif indexing and sequence comparison. The pipeline combines FASTA preprocessing, hash-based occurrence indexing, divide-and-conquer partitioning, multiprocessing, frequency and support filtering, LCS similarity, and Smith–Waterman local alignment. The supplied project documentation reports 20 passing unit tests and successful recovery of planted motifs in a synthetic 30-sequence dataset. These results support the educational and algorithmic purpose of the implementation but do not establish biological significance or universal performance gains. The system provides a foundation for further work on memory-efficient indexes, statistically grounded motif analysis, and scalable alignment.

## IX. DESIGN CONSIDERATIONS AND CORRECTNESS

### A. Sequence Normalization and Record Identity

Input normalization is part of the algorithmic contract because it determines which characters participate in a k-mer and how positions are interpreted. The parser must distinguish record headers from residue lines, concatenate wrapped sequence lines, and preserve a stable identifier for each record. Case normalization ensures that lowercase and uppercase representations of the same residue are treated consistently. Whitespace introduced by line wrapping should not create artificial characters. If the implementation removes digits or gap characters, this behavior should be documented because removal changes coordinates relative to the original file. In a scientific workflow, a stronger implementation would preserve a mapping from normalized coordinates to original coordinates so that every reported hit can be traced to the source record.

### B. Alphabet Validation

DNA and protein sequences use different alphabets. Alphabet detection should therefore be explicit and deterministic, especially when a short sequence could be interpreted as either type. Ambiguous nucleotide symbols are common in real sequence records, and rejecting or retaining them changes the search space. The current project applies the validation rules described in its documentation; it does not claim a comprehensive ambiguity model. A future version should expose an explicit alphabet setting and a policy for ambiguous residues, such as reject, split, or treat as a literal symbol. The selected policy should be recorded with every result so that repeated analyses are comparable.

### C. Correctness of Chunk Boundaries

Let a sequence have length n and let the maximum requested motif length be kmax. Valid start positions for a particular k lie in the inclusive range from zero to n−k. If a chunk owns a half-open interval [a,b) of start positions, it may need access to sequence characters through b+kmax−2 to construct the final window beginning at b−1. This is why text overlap and task ownership must be treated separately. Overlap makes characters available; it does not grant ownership of additional starts. If the overlap itself is counted by neighboring tasks, motifs can be duplicated. Assigning disjoint start ranges avoids this error and makes the merge operation a union of occurrence lists rather than a deduplication guess.

### D. Merge Invariants

Each partial map represents occurrences found by a specific set of disjoint start positions. The merge operation must preserve the motif key, sequence identifier, and each occurrence position. A useful invariant is that merging two maps over disjoint ranges produces the same result as scanning the union of those ranges directly. Position lists should be sorted before export to make results stable across worker scheduling orders. Tests should compare complete mappings rather than only total motif counts, because a count can be correct even when one position is missing and another

is duplicated. The supplied project documentation describes equivalence tests between a plain scanner and the divide-and-conquer implementation, including very small chunks.

### E. Deterministic Reporting

Parallel tasks may complete in different orders on different runs. Output order should therefore be determined by explicit sorting criteria rather than worker completion order. Ranking can prioritize occurrence count, sequence support, or motif length; ties need a stable secondary key, such as lexical motif order. Deterministic output simplifies regression testing, code review, and comparison of exported CSV or JSON files. The report should also include the values of kmin, kmax, thresholds, and execution mode because these settings affect the set of returned candidates.

## X. EVALUATION METHODOLOGY

### A. Functional Validation

Functional validation checks whether each module satisfies its defined contract. Parser tests should cover multiple FASTA records, wrapped sequence lines, mixed case, empty records, invalid characters, and duplicate identifiers. Motif tests should include sequences shorter than k, motifs at the first and last valid positions, repeated overlapping occurrences, and motifs spanning a chunk boundary. Alignment tests should cover identical strings, no shared characters, repeated characters, and local matches surrounded by unrelated prefixes and suffixes. The documented test suite reports 20 passing tests; this article summarizes those results as supplied rather than asserting a new test execution.

### B. Cross-Implementation Equivalence

A simple non-parallel scan is valuable as a reference implementation. The divide-and-conquer scanner can be compared against it over different recursion leaf sizes, while multiprocessing can be compared against the same expected map. Such checks are particularly important because concurrency bugs may appear only for certain chunk sizes or task orders. Equality should include motif keys, sequence identifiers, exact positions, and counts. For alignment, tests should check both the score and the traceback result against small examples whose optimal solution can be inspected manually.

### C. Benchmark Design

A meaningful performance experiment separates algorithmic work from measurement overhead. The sequential and parallel modes should receive the same sequences, k range, thresholds, and chunk definitions. The timer should specify whether process-pool startup and shutdown are included. Multiple trials help expose variability caused by operating-system scheduling, CPU frequency, memory pressure, and cache effects. The median is often a useful summary for skewed timing distributions, while minimum time can understate real application latency. A benchmark report should include input residues, number of records, k range, chunk size, worker count, Python version, CPU model, available memory, and whether the machine has multiple physical or logical cores.

### D. Performance Metrics

Let Tseq denote elapsed time for sequential execution and Tpar denote elapsed time for the parallel execution. Speedup is S=Tseq/Tpar. For p worker processes, parallel efficiency is E=S/p. A speedup greater than one means the parallel run was faster for the measured configuration; a value below one indicates that overhead outweighed the benefit. Efficiency close to one is uncommon for Python workloads that serialize substantial data or perform a merge phase. Since the supplied documentation does not provide a complete repeated-trial dataset and hardware profile, this paper does not provide fabricated timing values. The current evidence supports a correctness-oriented performance methodology rather than a universal scalability claim.

### E. Evaluation Boundaries

The synthetic planted-motif experiment provides a known target for checking exact recovery. It does not establish sensitivity or specificity on natural biological data, because random background composition and planted substrings do not model the full range of repeats, mutation, sequencing errors, and biological constraints. A stronger scientific evaluation would use curated datasets, define a ground-truth motif set, compare results with established tools, and report precision, recall, false-discovery behavior, and runtime. Statistical motif discovery tools such as STREME and SEA address significance-related questions that are outside the present exact-counting objective [3], [4].

## XI. PRACTICAL USE AND OUTPUT INTERPRETATION

### A. Command-Line Workflow

The command-line interface supports repeatable analysis from a FASTA input path and exposes search parameters such as the minimum and maximum motif length, sequence-support threshold, sorting mode, and optional alignment. A typical workflow is to first run the bundled example, inspect the output format, then analyze a custom FASTA file with a deliberately narrow k range. Narrow parameter ranges are easier to interpret and can reduce memory consumption. The benchmark command should be run separately from the analysis command so that timing output is not confused with motif-result output. Exact command names and accepted options should be checked against the supplied source files before use.

### B. Dashboard Workflow

The local dashboard presents an alternative for users who prefer interactive controls. A typical analysis begins by supplying a dataset, choosing k bounds and thresholds, selecting the execution mode, and submitting the request. The interface then presents a ranked motif table and may expose position details, sequence comparisons, and export controls. CSV is suitable for tabular downstream analysis, whereas JSON is better suited to retaining nested data such as positions grouped by sequence. The dashboard is described as a local educational interface; exposing it to a public network would require authentication, request limits, error handling, and security review.

### C. Interpreting Frequency and Support

Occurrence count and sequence support describe different aspects of recurrence. A motif can have a high count because it repeats many times in one sequence while appearing in few records. Another motif can have a lower total count but be present across many sequences. Reporting both metrics gives a more complete view. Position lists help the user inspect clustering and overlap, but the presence of repeated windows should not be interpreted as a functional binding site without additional evidence. A motif that differs by one residue from another may appear as a separate exact key; approximate matching requires a different algorithm.

### D. Interpreting Alignment Scores

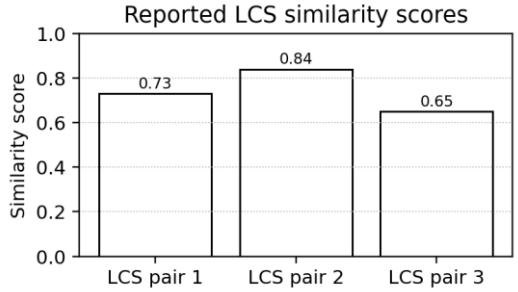



Fig. 4. Reported LCS similarity scores for three sequence comparisons.

LCS similarity measures the length of an order-preserving common subsequence relative to the lengths of both input sequences. It allows gaps in the sense that intervening characters may be skipped, but it does not use a biological substitution matrix. Smith–Waterman instead optimizes a local score under the configured match, mismatch, and gap parameters. The reported local identity applies to the aligned region, not necessarily to the full sequences. Two outputs can therefore differ without contradiction: one describes ordered shared symbols, while the other describes the best-scoring local alignment under a scoring model.

### E. Responsible Use

The framework is appropriate for education, exploratory analysis, and validating string-processing algorithms. It should not be used to infer diagnosis, pathogenicity, gene regulation, or protein function from repeated strings alone. Real biological interpretation requires relevant reference datasets, statistical controls, domain-specific scoring, and expert review. Any results shared in a report should distinguish between implementation behavior, synthetic examples, and claims supported by biological evidence.

## XII. FUTURE RESEARCH DIRECTIONS

### A. Compact K-mer Representations

For DNA sequences, short k-mers can sometimes be encoded using two bits per nucleotide, while rolling hashes can update a window representation without allocating a new substring for every start. These techniques may reduce memory traffic and substring-construction overhead, although collision handling and ambiguous symbols require care. The literature on exact associative structures and space-efficient k-mer indexes provides relevant design alternatives [5], [7], [8].

### B. Statistical Motif Discovery

A next step is to supplement exact recurrence with a statistical model that estimates whether a candidate motif is enriched relative to an appropriate background. Such a module would need explicit assumptions, background construction, multiple-testing treatment, and significance reporting. STREME and SEA are relevant references in the supplied collection for motif discovery and enrichment [3], [4]. Integration should preserve a clear distinction between the current exact-counting output and statistical evidence.

### C. Scalable Alignment

The current full dynamic-programming tables are suitable for short comparisons but become expensive for long sequences. Rolling-row LCS can reduce memory when only the length is required; traceback needs additional storage or a reconstruction strategy. For local alignment, tiled computation, affine-gap scoring, optimized native libraries, GPU/FPGA approaches, and actor-based scheduling are directions represented in the supplied literature [12]–[15]. Any optimization should be validated against the baseline recurrence on small inputs before performance claims are made.

### D. Reproducible Benchmarking

Future evaluation should include repeated trials across input sizes and worker counts, record peak memory, and separate parsing, search, merge, alignment, and export time. Scaling experiments should include both many short sequences and fewer long sequences because the task distribution affects scheduling overhead. Comparisons with established tools should use identical input datasets and explicitly state which outputs are comparable. Publishing scripts, synthetic-data seeds, parameter files, and machine details would improve reproducibility.

## XIII. THREATS TO VALIDITY

### A. Construct Validity

The term motif can refer to different computational objects. In this implementation, a reported motif is an exact substring of a selected length that passes occurrence and support filters. In biological research, a motif may instead be represented by a position-weight matrix, a consensus pattern with allowed variation, a structural pattern, or a statistically enriched sequence model. Therefore, the paper uses the phrase exact motif or recurring k-mer when describing outputs from the current implementation. This wording avoids suggesting that the engine discovers a functional regulatory element or a statistically significant motif in the broader biological sense.

### B. Internal and External Validity

Internal validity depends on whether the search implementation counts every valid start exactly once, merges all worker outputs, and applies filters consistently. The supplied documentation reports tests for these properties, but the test results are not an independent audit of every possible input. External validity is limited by the use of short examples and synthetic planted motifs. Sequence composition, repeats, ambiguity symbols, and length

distributions in real datasets may differ substantially. Consequently, the reported motif counts should be interpreted as evidence for behavior on the documented examples rather than evidence that the same ranking will hold in another dataset.

### C. Measurement Validity

Performance measurements are meaningful only when their scope is clear. A timer around the scan alone does not represent the latency of a dashboard request if parsing, process startup, merging, alignment, and serialization occur outside the timer. Likewise, the number of worker processes does not necessarily equal the number of cores available to the process. The supplied project material does not provide enough repeated timing data and machine metadata to support a general speedup claim. This paper therefore reports the benchmark design and known caveats, not fabricated measurements.

## XIV. REQUIREMENTS TRACEABILITY

### A. Functional Traceability

The main functional requirements map directly to the pipeline. FASTA ingestion and alphabet checks are handled by preprocessing. Exact motif discovery and position recording are handled by the k-mer index. Chunk scheduling and partial-result aggregation are handled by divide-and-conquer and multiprocessing modules. Filtering, sorting, and exports are handled by analysis. LCS and Smith–Waterman routines provide pairwise comparison. The CLI and local dashboard expose the same core operations. This decomposition makes it possible to test each behavior in isolation and then test integration across the complete request path.

### B. Non-Functional Traceability

Correctness is addressed through reference-scan equivalence and boundary tests. Reproducibility is supported by deterministic output ordering and documented synthetic examples. Portability follows from the use of Python and its standard library. Maintainability is supported by module separation. Performance is addressed through a benchmark that reports elapsed time and result equality, although the available evidence is not sufficient to establish a universal scaling curve. Security is limited by the intended local educational use; a public deployment would need request-size limits, authentication where appropriate, and explicit resource quotas.

### C. Requirement-to-Evidence Summary

The supplied project documentation reports 20 passing tests, including input validation, search equivalence, chunk-boundary handling, sequential/parallel consistency, alignment behavior, and web API validation. The bundled example provides known recurring motifs and positions, while the planted-motif dataset provides a controlled recovery scenario. Together, these materials provide functional evidence for the project's objectives. They do not, however, replace an independent code audit, a real biological benchmark, or a repeated multi-core performance study.

## XV. CONCLUDING OBSERVATIONS

### A. Summary of Contributions

The work brings together a position-aware exact k-mer index, a divide-and-conquer scan strategy, optional multiprocessing, frequency-based analysis, LCS similarity, Smith–Waterman local alignment, and interfaces for command-line and browser-based use. The architecture emphasizes a shared core engine and explicit merge semantics. These design choices make the project useful for understanding how algorithmic primitives can be combined into a practical data-analysis tool.

### B. Final Perspective

The project is best viewed as a transparent baseline and teaching platform. Its output is auditable because motif keys, counts, sequence support, and positions are exposed. Its boundaries are equally important: exact recurrence is not a substitute for statistical motif discovery, and local alignment scores depend on their scoring scheme. The next development stage should prioritize reproducible real-data evaluation, efficient storage, statistical interpretation, and alignment scalability before claiming suitability for large-scale research workflows.

## REFERENCES


[1] Fatma A. Hashim, Mai S. Mabrouk, W. Al-Atabany, "Review of Different Sequence Motif Finding Algorithms,", Avicenna Journal of Medical Biotechnology, 2019.

[2] C. Marchet, C. Boucher, S. Puglisi, P. Medvedev, Mikaël Salson, R. Chikhi, "Data structures based on k-mers for querying large collections of sequencing data sets,", Genome Research, 2019, doi: 10.1101/866756.

[3] T. Bailey, "STREME: Accurate and versatile sequence motif discovery,", bioRxiv, 2020, doi: 10.1101/2020.11.23.394619.

[4] T. Bailey, Charles E. Grant, "SEA: Simple Enrichment Analysis of motifs,", bioRxiv, 2021, doi: 10.1101/2021.08.23.457422.

[5] C. Marchet, Mael Kerbiriou, A. Limasset, "BLight: efficient exact associative structure for k-mers,", Bioinformatics, 2021, doi: 10.1093/bioinformatics/btab217.

[6] Jarno N. Alanko, Jaakko Vuohtoniemi, T. Mäklin, S. Puglisi, "Themisto: a scalable colored k-mer index for sensitive pseudoalignment against hundreds of thousands of bacterial genomes,", Bioinformatics, 2023, doi: 10.1093/bioinformatics/btad233.

[7] O. Sladký, P. Veselý, K. Břinda, "FroM Superstring to Indexing: a space-efficient index for unconstrained k-mer sets using the Masked Burrows-Wheeler Transform (MBWT),", Bioinformatics Advances, 2025, doi: 10.1093/bioadv/vbaf290.

[8] Subhankar Roy, A. Mukhopadhyay, "A Randomized Optimal k-mer Indexing Approach for Efficient Parallel Genome Sequence Compression.,", Gene, 2024, doi: 10.1016/j.gene.2024.148235.

[9] Arpan Kumar, Sarbajit Manna, "Graphic user interface based implementation of longest common subsequence problem in DNA sequencing,", SEVENTH INTERNATIONAL SYMPOSIUM ON NEGATIVE IONS, BEAMS AND SOURCES (NIBS 2020), 2021, doi: 10.1063/5.0057916.

[10] S. Singh, Dharanya T, Aarthi N, N. S, "A Dynamic MPI-Based Memory-Efficient Framework for Longest Common Subsequence Computation on Massive DNA Sequence,", International Journal of Innovative Science and Research Technology, 2025, doi: 10.38124/ijisrt/25may1297.

[11] Z. Kong, Shiwei Wei, Wen-Ming Huang, Rui Yang, Zhenrong Deng, "An Efficient Parallel and Adaptive Algorithm for Large-Scale Multiple Longest Common Subsequence Problems,", IEEE Access, 2026, doi: 10.1109/access.2026.3659505.

[12] M. Pałkowski, W. Bielecki, "Parallel Tiled Codes Implementing the Smith–Waterman Alignment Algorithm for Two and Three Sequences,", Journal of Computational Biology, 2018, doi: 10.1089/cmb.2018.0006.

[13] F. N. Muhamad, R. Ahmad, S. M. Asi, M. Murad, "Performance Analysis Of Needleman-Wunsch Algorithm (Global) And Smith-Waterman Algorithm (Local) In Reducing Search Space And Time For Dna Sequence Alignment,",

Journal of Physics: Conference Series, 2018, doi: 10.1088/1742-6596/1019/1/012085.

[14] Asmaa G. Seliem, Hesham F. A. Hamed, Wael Abouelwafa, "MapReduce Model Using FPGA Acceleration for Chromosome Y Sequence Mapping,", IEEE Access, 2021, doi: 10.1109/access.2021.3085997.

[15] Reza Rafati Bonab, A. Jamali, Kyle Klenk, Mohammad Mahdi Moayeri, Raymond J. Spiteri, "SW-actors: accelerating the Smith–Waterman algorithm via actors,", Bioinformatics Advances, 2025, doi: 10.1093/bioadv/vbaf173.

[16] Jie-Fu Li, Zi-Yuan Wang, Xuwei Fan, Ruijie Yao, Guoqing Zhang, Rui Fan, Ze-Feng Wang, "Rapid multiple protein sequence search by parallel and heterogeneous computation,", Bioinformatics, 2024, doi: 10.1093/bioinformatics/btae151.

[17] Yanming Wei, Quan Zou, Furong Tang, Liang Yu, "WMSA: a novel method for multiple sequence alignment of DNA sequences,", Bioinformatics, 2022, doi: 10.1093/bioinformatics/btac658.

[18] R. Lebsir, Abdesslem Layeb, F. Tahi, "A Greedy Clustering Algorithm for Multiple Sequence Alignment,", Int. J. Cogn. Informatics Nat. Intell., 2021, doi: 10.4018/ijcini.20211001.oa41.

[19] Jeremy W. Schroeder, Vivian Ramirez, Michael B. Wolfe, L. Freddolino, "ShapeME: A tool and web front-end for de novo discovery of structural motifs underpinning protein-DNA interactions,", Journal of molecular biology, 2026, doi: 10.1016/j.jmb.2026.169810.